# BABU BANARASI DAS UNIVERSITY, LUCKNOW

School of Computer Applications

SESSION: 2025-2026

PROJECT REPORT

ON

# BIG DATA FUNDAMENTALS

(Course Code: BCADSN14201)

PROJECT TITLE:

## Sales Data Analysis Using Hadoop MapReduce and Hive

SUBMITTED TO

**Mr. Vikas**

Faculty, School of Computer Applications

SUBMITTED BY


Group Members:

- Anuj Srivastava
- Ankit Patel
- Aryan Pratap Singh
- Anurag Yadav


SUBMISSION DATE

# PROJECT

## Sales Data Analysis using Hadoop MapReduce and Hive

- **Architecture:**
  The architecture of this project is based on the Hadoop ecosystem, where HDFS is used for storage, MapReduce is used for processing, and Hive is used for data analysis in a distributed environment

- **Data Flow:**

  - **Raw Dataset → HDFS → MapReduce Processing → Processed Data → Hive Analysis → Insights**

- **Explanation:**

  - Sales data is stored in HDFS.
  - MapReduce processes the data and calculates total sales.
  - The output is stored again in HDFS.
  - Hive is used to analyze the data and get results.

- **Tools Used:**

  - **HDFS** – Used for storing the sales dataset in a distributed manner
  - **MapReduce** – Used to process data and calculate product-wise total sales
  - **Hive** – Used to perform analysis using SQL queries
  - **Linux Terminal** – Used to execute commands
  - **ChatGPT** – Used for guidance and understanding(Generative AI tool)

- **Technologies:**

  - Python – Used to write Mapper and Reducer scripts
  - CSV Dataset – Used to store sales data in structured format
  - Hadoop Ecosystem – Used for distributed storage and processing

- **Use of Generative AI in the Project:**

**• In this project, Generative AI tools were utilized to support development and enhance understanding of Big Data concepts.**

- **Areas Where AI Was Used:**

  1. **Code Generation**
     - Generative AI was used to generate MapReduce programs (mapper and reducer).

  2. **Debugging**
     - Errors in code were identified and fixed using AI assistance.

  3. **Query Writing**
     - Hive queries were generated and optimized using AI.

  4. **Concept Understanding**
     - AI helped in understanding:
       - HDFS architecture
       - MapReduce workflow
       - Hive operations

- **Example Prompt Used:**

  Write a Hadoop MapReduce program to process sales dataset and extract product-wise information.

- **Benefits of Using AI:**
  - **Faster development**
  - **Reduced coding errors**
  - **Better understanding of concepts**
  - **Improved efficiency**

## Dataset Source:

- The dataset used in this project is a structured sales dataset created for academic purposes to demonstrate data processing and analysis using Hadoop technologies.

### ✓ Purpose of Dataset:

- **The dataset is used to:**
  - **Analyze product-wise sales data.**
  - **To calculate total sales using Hadoop MapReduce.**
  - **Perform data analysis using Hive queries.**
  - **Generate meaningful insights such as top-selling products.**

## ➢ Sample Data:

```
orderID,product,category,region,sales,quantity,discount,profit
1,A,Furniture,North,100,2,0.1,20
2,B,Technology,West,200,3,0.2,50
3,A,Furniture,South,150,1,0.0,30
4,C,Office,East,300,4,0.3,70
5,B,Technology,North,100,2,0.1,25
6,D,Furniture,West,250,3,0.2,40
7,E,Office,South,400,5,0.0,90
8,A,Furniture,East,200,2,0.1,35
9,C,Office,North,150,1,0.0,20
10,B,Technology,South,300,4,0.2,60
```

## Conclusion

- **In this project, a complete Big Data processing workflow was implemented using the Hadoop ecosystem.**

- **The sales dataset was stored in HDFS and processed using MapReduce to calculate product-wise total sales.**

- **The Hive was used to analyze the processed data and extract meaningful insights such as top-selling products.**

- **This project helped in understanding distributed data processing and analysis techniques.**

- **Generative AI assisted in code development, debugging, and concept understanding throughout the project.**

**Generative AI was used not only for code generation but also as a learning assistant to understand Big Data workflows and improve overall efficiency.**

# Problem Statement :-1

## Sales Data Analysis using Hadoop Streaming.

- **Aim: - " To analyze sales dataset and calculate product-wise total sales using Hadoop MapReduce."**

**Step-01:** - Create **a "CSV" file.**

- **Cat > sales.csv**

```
[cloudera@quickstart ~]$ cat sales.csv
orderID,product,category,region,sales,quantity,discount,profit
1,A,Furniture,North,100,2,0.1,20
2,B,Technology,West,200,3,0.2,50
3,A,Furniture,South,150,1,0.0,30
4,C,Office,East,300,4,0.3,70
5,B,Technology,North,100,2,0.1,25
6,D,Furniture,West,250,3,0.2,40
7,E,Office,South,400,5,0.0,90
8,A,Furniture,East,200,2,0.1,35
9,C,Office,North,150,1,0.0,20
10,B,Technology,South,300,4,0.2,60
```

**Step-02:- Create a directory**

- **hdfs dfs -mkdir /user/cloudera/sales**
- **hdfs dfs -ls /user/cloudera**

```
[cloudera@quickstart ~]$ hdfs dfs -mkdir /user/cloudera/sales
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera
Found 4 items
drwxr-xr-x   - cloudera cloudera          0 2026-03-23 21:26 /user/cloudera/anuj
drwxr-xr-x   - cloudera cloudera          0 2026-04-08 09:56 /user/cloudera/anuj
srivastava
drwxr-xr-x   - cloudera cloudera          0 2026-04-10 02:36 /user/cloudera/inpu
t
drwxr-xr-x   - cloudera cloudera          0 2026-04-23 00:18 /user/cloudera/sale
s
[cloudera@quickstart ~]$
```

**Step-03:- Load the "CSV" on Cloudera's Server**

- **hdfs dfs -put sales.csv /user/cloudera/sales**
- **hdfs dfs -ls /user/cloudera/sales**

```
[cloudera@quickstart ~]$ hdfs dfs -put sales.csv /user/cloudera/sales
```

```
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/sales
Found 1 items
-rw-r--r--   1 cloudera cloudera        384 2026-04-23 00:22 /user/cloudera/sales/sal
es.csv
[cloudera@quickstart ~]$
```

**Step-04:-** Create a mapper file using “nano”

- **#!/usr/bin/env python**
- **import sys**

- **for line in sys.stdin:**
- **if "orderID" in line:**
- **continue**

- **data = line.strip().split(',')**

- **product = data[1]**
- **sales = int(data[4])**

- **print(f"{product}\t{sales}")**

```
import sys

for line in sys.stdin:
if "orderID" in line:
continue

data = line.strip().split(',')

product = data[1]
sales = int(data[4)



print(f"{product}\t{sales}")
```

**Step-05:-** Create a reducer file using “nano”

- **#!/usr/bin/env python**
- **import sys**

- **current = None**
- **total = 0**

- **for line in sys.stdin:**
- **product, value = line.strip().split('\t')**
- **value = int(value)**

- **if current == product:**
- **total += value**
- **else:**
- **if current:**
    - **print(f"{current}\t{total}")**
- **current = product**
- **total = value**

- **if current:**
- **print(f"{current}\t{total}")**

```
current = None
total = 0

for line in sys.stdin:
        product,value = line.strip().split('\t')
        value = int(value)

        if current == product:
        total += value

        else:
        if current:
                print(f"{current}\t{total}")
        current = product
        total = value

if current:
        print(f"{current}\t{total}")
```

## Step-06:- Make executable

- **chmod +x mapper.py reducer.py**

```
[cloudera@quickstart ~]$ chmod +x mapper.py reducer.py
[cloudera@quickstart ~]$ ls
anuj                Documents                    Pictures            SalesMapper.java
anuj2.txt           Downloads                    Public              sales_project
anuj4.txt           eclipse                      reducer             SalesReducer.class
anuj.txt            enterprise-deployment.json   reducer.py          SalesReducer.java
bigdata             express-deployment.json      reducer.py.save     SampleSuperstore.csv
big_data_demo       kerberos                     reducer.py.save.1   superstore.csv
cloudera-manager    lib                          sales.csv           Templates
cm_api.py           mapper.py                    SalesDriver.class   Videos
data.txt            map_reduce                   SalesDriver.java    word.txt
data.txty           Music                        sales.jar           workspace
Desktop             parcels                      SalesMapper.class
[cloudera@quickstart ~]$ chmod +x mapper.py reducer.py
[cloudera@quickstart ~]$ ls
anuj                Documents                    Pictures            SalesMapper.java
anuj2.txt           Downloads                    Public              sales_project
anuj4.txt           eclipse                      reducer             SalesReducer.class
anuj.txt            enterprise-deployment.json   reducer.py          SalesReducer.java
bigdata             express-deployment.json      reducer.py.save     SampleSuperstore.csv
big_data_demo       kerberos                     reducer.py.save.1   superstore.csv
cloudera-manager    lib                          sales.csv           Templates
cm_api.py           mapper.py                    SalesDriver.class   Videos
data.txt            map_reduce                   SalesDriver.java    word.txt
data.txty           Music                        sales.jar           workspace
Desktop             parcels                      SalesMapper.class
[cloudera@quickstart ~]$ ls -l
total 1364
drwxrwxr-x 2 cloudera cloudera    4096 Mar 24 09:57 anuj
-rw-rw-r-- 1 cloudera cloudera     103 Apr  8 09:43 anuj2.txt
```

## Step-07:- Copy "sales.csv" /home/cloudera to /home/cloudera/mapreduce

- **cp /home/cloudera/movie.csv /home/cloudera/mapreduce/movie.csv**

```
[cloudera@quickstart ~]$ cp /home/cloudera/sales.csv /home/cloudera/mapreduce/sales.csv
[cloudera@quickstart ~]$ ls /home/cloudera/mapreduce
sales.csv
```

## Step-08:- Local test

- **cat sales.csv | python mapper.py**
- **cat sales.csv | python mapper.py | sort | python reducer.py**

```
[cloudera@quickstart ~]$ cat sales.csv | python mapper.py
A       100
B       200
A       150
C       300
B       100
D       250
E       400
A       200
C       150
B       300
[cloudera@quickstart ~]$ cat sales.csv | python mapper.py | sort | python reducer.py
A       450
B       600
C       450
D       250
E       400
[cloudera@quickstart ~]$
```

## Step-09:- Run Hadoop Job

- **hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar **
  **-files mapper.py,reducer.py **
  **-input /user/cloudera/sales/sales.csv **
  **-output /user/cloudera/sales/output **
  **-mapper "python mapper.py" **
  **-reducer "python reducer.py"**

```
[cloudera@quickstart ~]$ hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \
> -files mapper.py,reducer.py \
> -input /user/cloudera/sales/sales.csv \
> -output /user/cloudera/sales/output \
> -mapper "python mapper.py" \
> -reducer "python reducer.py"
packageJobJar: [] [/usr/lib/hadoop-mapreduce/hadoop-streaming-2.6.0-cdh5.13.0.jar] /tmp/strea
mjob7946550624506827885.jar tmpDir=null
26/04/23 01:58:16 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/23 01:58:16 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/23 01:58:17 INFO mapred.FileInputFormat: Total input paths to process : 1
26/04/23 01:58:17 INFO mapreduce.JobSubmitter: number of splits:2
26/04/23 01:58:18 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1776930616928_0
002
26/04/23 01:58:19 INFO impl.YarnClientImpl: Submitted application application_1776930616928_0
002
26/04/23 01:58:19 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:80
88/proxy/application_1776930616928_0002/
26/04/23 01:58:19 INFO mapreduce.Job: Running job: job_1776930616928_0002
26/04/23 01:58:30 INFO mapreduce.Job: Job job_1776930616928_0002 running in uber mode : false
26/04/23 01:58:30 INFO mapreduce.Job:  map 0% reduce 0%
26/04/23 01:58:37 INFO mapreduce.Job:  map 50% reduce 0%
26/04/23 01:58:38 INFO mapreduce.Job:  map 100% reduce 0%
26/04/23 01:58:43 INFO mapreduce.Job:  map 100% reduce 100%
26/04/23 01:58:44 INFO mapreduce.Job: Job job_1776930616928_0002 completed successfully
26/04/23 01:58:45 INFO mapreduce.Job: Counters: 49
        File System Counters
                FILE: Number of bytes read=86
```

## Step-10:- View Output

- **hdfs dfs -cat /user/cloudera/sales/output/part-00000**
- **hdfs dfs -ls /user/cloudera/sales**

```
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/sales
Found 2 items
drwxr-xr-x   - cloudera cloudera          0 2026-04-23 01:58 /user/cloudera/sales/output
-rw-r--r--   1 cloudera cloudera        384 2026-04-23 00:22 /user/cloudera/sales/sales.csv
[cloudera@quickstart ~]$ hdfs dfs -cat /user/cloudera/sales/output/part-00000
A       450
B       600
C       450
D       250
E       400
[cloudera@quickstart ~]$
```

## Problem Statement 2 :-

**To perform data analysis on processed sales dataset using Hive.**

- **Aim: - "To analyze movie data and find top-rated sales using Hive queries."**

**Step-01:- Open Hive**

- **hive**

```
[cloudera@quickstart mapreduce]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive>
```

**Step-02:- Create table on hive**

- **CREATE TABLE sales_output (**
  **product STRING,**
  **total_sales INT**
  **);**

```
hive> CREATE TABLE sales_output (
    > product STRING,
    > total_sales INT
    > );
OK
Time taken: 2.415 seconds
```

**Step-03:- Load data on the table in movie_avg**

- **LOAD DATA INPATH '/user/cloudera/sales/output' INTO TABLE sales_output;**

```
hive> LOAD DATA INPATH '/user/cloudera/sales/output' INTO TABLE sales_output;
Loading data to table default.sales_output
Table default.sales_output stats: [numFiles=1, totalSize=30]
OK
Time taken: 0.794 seconds
hive>
```

## Step-04:- View Data

- **SELECT * FROM sales_output;**

```
hive> SELECT * FROM sales_output;
OK
A        450      NULL
B        600      NULL
C        450      NULL
D        250      NULL
E        400      NULL
Time taken: 0.562 seconds, Fetched: 5 row(s)
hive>
```

## 🔥 Top selling product (highest sales)

**SELECT ***
**FROM sales_output**
**ORDER BY total_sales DESC**
**LIMIT 1;**

```
hive> SELECT *
    > FROM sales_output
    > ORDER BY total_sales DESC
    > LIMIT 1;
Query ID = cloudera_20260423022727_f2472d7e-b188-4ab4-842a-c12e4134fae9
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1776930616928_0003, Tracking URL = http://quickstart.cloudera:8088/proxy/a
pplication_1776930616928_0003/
Kill Command = /usr/lib/hadoop/bin/hadoop job  -kill job_1776930616928_0003
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-23 02:28:02,674 Stage-1 map = 0%,  reduce = 0%
2026-04-23 02:28:10,130 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 2.25 sec
2026-04-23 02:28:17,498 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 3.84 sec
MapReduce Total cumulative CPU time: 3 seconds 840 msec
Ended Job = job_1776930616928_0003
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 3.84 sec   HDFS Read: 6799 HDFS Write: 9 S
UCCESS
Total MapReduce CPU Time Spent: 3 seconds 840 msec
OK
E        400      NULL
Time taken: 26.51 seconds, Fetched: 1 row(s)
hive>
```

## 🚀 Top 3 products

- **SELECT * FROM sales_output ORDER BY total_sales DESC LIMIT 3;**

```
hive> SELECT * FROM sales_output ORDER BY total_sales DESC LIMIT 3;
Query ID = cloudera_20260423023434_c1d9dda3-4137-4326-9e44-77eae21f2fee
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducer.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1776930616928_0004, Tracking URL = http://quickstart.cloudera:8088/proxy/a
pplication_1776930616928_0004/
Kill Command = /usr/lib/hadoop/bin/hadoop job  -kill job_1776930616928_0004
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-23 02:35:05,747 Stage-1 map = 0%,  reduce = 0%
2026-04-23 02:35:12,047 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 1.45 sec
2026-04-23 02:35:18,393 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 3.7 sec
MapReduce Total cumulative CPU time: 3 seconds 700 msec
Ended Job = job_1776930616928_0004
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 3.7 sec   HDFS Read: 6862 HDFS Write: 27 S
UCCESS
Total MapReduce CPU Time Spent: 3 seconds 700 msec
OK
E       400     NULL
D       250     NULL
C       450     NULL
Time taken: 22.519 seconds, Fetched: 3 row(s)
hive>
```

## 🔹 Output:

## Hive successfully displayed:

🔹 The output of the project was successfully generated using Hadoop MapReduce and analyzed using Hive. The processed data shows the total sales for each prod

### 📊 Sample Output (MapReduce Result

A 450
B 600
C 450
D 250
E 400

### 📈 Hive Query Output (Top Sales)

| Product | Total_Sales |
|---|---|
| B | 600 |
| E | 400 |
| A | 450 |

In this practical, Hive was employed to efficiently analyze the processed sales dataset and derive meaningful insights in a structured manner.